

RISC_AXI Mini MCU 발표 대본 + 코드 공부 가이드

작성 기준: 현재 `slide.html` 22장 구성, `Project/risc_axi` 최신 RTL/ASM 구조 기준.

이 문서는 PPT 발표 직전에 보는 발표자용 자료다. 각 슬라이드마다 아래 네 가지를 정리했다.

- 발표 대본: 실제로 말할 수 있는 문장
- 핵심 이해: 발표 전에 머릿속에 잡아야 하는 개념
- 코드 공부 위치: 어느 파일의 어느 부분을 보면 되는지
- 질문 대비: 발표 후 질문이 들어왔을 때 답할 포인트

0. 발표 전체 흐름

발표의 큰 문장은 하나로 잡으면 된다.

STM32F103 같은 MCU 구조를 참고해서 출발했지만, 실제 구현은 직접 만든 RV32I core를 중심으로 AXI-Lite 제어 경로, APB peripheral, AXI-Stream DMA 데이터 경로를 분리한 mini MCU SoC로 재설계했다.

발표 순서는 다음 흐름이 자연스럽다.

구간	슬라이드	말할 핵심
도입	1-4	무엇을 만들었고, 왜 STM32F103을 참고했는가
시스템 구조	5-9	<code>SocTop</code> , AXI-Lite/APB, memory map, AXI-Stream 역할
core/interrupt	10-15	RV32 pipeline, CSR, trap, PLIC, software vector dispatch
응용 시나리오	16-18	PC UART -> master FPGA DMA -> SPI -> slave FPGA -> UART -> PC
검증/마무리	19-22	파형 자리, demo 자리, 결과와 느낀점

중요한 정정 포인트:

- 이 발표의 메인 시나리오는 `RGB IRQ forward` 계열이다.
- 최신 ROM은 `Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S` 를 기준으로 본다.
- `final_source/DMA_HW_SW_EXPLANATION.md` 와 일부 예전 문서는 AHB 또는 이전 final demo 설명이 섞여 있으므로, 지금 PPT에는 현재 AXI 구조에 맞게 바꿔서 말해야 한다.
- PLIC이 handler 주소를 직접 주는 것이 아니다. PLIC은 claim ID를 주고, ROM의 software vector table이 handler 주소를 고른다.
- routine은 ARM NVIC든 현재 RISC-V 설계든 결국 ROM/Flash code 영역에 있다. 차이는 "누가 handler 주소를 선택하느냐"다.

1. Slide 1 - Mini MCU Project

발표 대본

이 프로젝트는 직접 만든 RV32I RISC-V core를 중심으로 mini MCU 형태의 SoC를 구성한 프로젝트입니다. 단순히 CPU만 만든 것이 아니라, CPU가 주변장치를 제어할 수 있도록 AXI-Lite bus와 APB peripheral subsystem을 붙였고, RGB image 같은 연속 byte data를 처리하기 위해 AXI-Stream DMA 경로까지 분리했습니다.

발표에서는 크게 세 가지를 보시면 됩니다. 첫째, custom core가 어떻게 SoC top에 연결되는지, 둘째, 제어 bus와 data stream을 왜 분리했는지, 셋째, interrupt와 DMA를 이용해서 RGB byte stream을 어떻게 처리했는지입니다.

핵심 이해

- Mini MCU 는 core 하나가 아니라 core, bus, memory map, peripheral, interrupt, firmware ROM이 같이 묶인 시스템이다.
- AXI-Lite 는 CPU가 register를 읽고 쓰는 제어 경로다.
- AXI-Stream 은 주소 없이 byte payload가 흐르는 데이터 경로다.
- 이 프로젝트의 발표 중심은 "직접 만든 RV32I core로 작은 MCU SoC를 구성하고, 이후 data stream demo까지 확장했다"는 흐름이다.

코드 공부 위치

위치	블 내용
Project/risc_axi/src/soc/SocTop.sv:17	SoC 본체 module 시작
Project/risc_axi/src/soc/SocTop.sv:177	Rv32Core instance
Project/risc_axi/src/soc/SocTop.sv:211	IcodeLocalRom instance
Project/risc_axi/src/soc/SocTop.sv:261	AxiLiteInterconnect1x3 instance
Project/risc_axi/src/soc/SocTop.sv:419	ApbSubsystem instance
slide.html:601	표지 slide title

질문 대비

- Q. CPU만 만든 프로젝트인가?
- A. 아니다. RV32I core가 중심이지만, 실제 목표는 core가 bus와 peripheral을 제어하는 mini MCU SoC 구조를 만드는 것이다.

2. Slide 2 - Table of Contents

발표 대본

발표는 프로젝트 목표에서 시작해서 STM32F103 데이터시트를 참고한 초기 방향을 먼저 설명하고, 그 다음 실제로 바뀐 SocTop 구조와 bus 구조를 설명하겠습니다. 이후에는 core와 interrupt, PLIC 구조를 보고, 마지막으로 RGB image stream 시나리오와 검증, 느낀점으로 마무리하겠습니다.

핵심 이해

- 목차는 "왜 이렇게 만들었는가"에서 "어떻게 동작하는가"로 넘어가야 한다.
- 3장과 4장은 구조 설명, 5장은 interrupt 설명, 6장은 실제 application flow, 7장은 검증과 회고다.

코드 공부 위치

위치	볼 내용
slide.html:609	Table of Contents
PPT_FINAL_STORYBOARD_RGB_IRQ.md	이전에 잡아 둔 최종 스토리보드
RISC_AXI_PRESENTATION_GUIDE.md	발표 큰 흐름과 문장 참고

질문 대비

Q. 왜 bus 이야기가 발표 중간에 큰 비중을 차지하나?

A. 이 프로젝트의 핵심이 CPU와 peripheral을 그냥 연결하는 것이 아니라, control transaction과 byte stream movement를 분리하는 것이기 때문이다.

3. Slide 3 - Project Goal: Little MCU Design

발표 대본

이 프로젝트의 첫 목표는 직접 만든 RV32I core를 중심으로 작은 MCU처럼 동작하는 SoC를 구성하는 것이었습니다. CPU 하나만 만든 것이 아니라, instruction ROM, SRAM, memory-mapped peripheral, interrupt controller까지 붙여서 실제 firmware가 주변장치를 제어할 수 있는 구조를 만들었습니다.

이후에는 이 mini MCU 구조 위에서 GPIO, UART, SPI, timer, PLIC 같은 peripheral을 제어하고, RGB image stream demo까지 확장했습니다. 그래서 이 slide에서는 먼저 "작은 MCU 전체를 직접 설계했다"는 점을 잡고 넘어가면 됩니다.

핵심 이해

- 핵심은 CPU 단품이 아니라 ROM, SRAM, bus, peripheral, interrupt가 묶인 mini MCU 구조다.
- STM32F103 같은 MCU block diagram을 참고했지만, 실제 구현은 custom RV32I 기반으로 직접 구성했다.
- DMA와 AXI-Stream은 이 little MCU 위에서 RGB byte stream을 처리하기 위한 확장 요소로 뒤에서 설명한다.

코드 공부 위치

위치	볼 내용
Project/risc_axi/src/soc/SocTop.sv:17	mini MCU SoC 본체
Project/risc_axi/src/soc/SocTop.sv:177	Rv32Core 연결
Project/risc_axi/src/soc/SocTop.sv:211	instruction ROM 연결
Project/risc_axi/src/soc/SocTop.sv:261	AXI-Lite interconnect 연결
Project/risc_axi/src/soc/SocTop.sv:419	APB peripheral subsystem 연결
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:247	firmware 초기화 시작

질문 대비

- Q. 이 프로젝트의 가장 기본 목표는 무엇인가?
- A. 직접 만든 RV32I core를 중심으로 ROM, SRAM, bus, peripheral, interrupt가 있는 작은 MCU SoC를 설계하는 것이다. DMA/RGB stream은 그 구조 위에 올린 응용 시나리오다.

4. Slide 4 - STM32F103 Reference & Redesign Direction

발표 대본

초기에는 STM32F103 데이터시트의 performance line block diagram을 보고 MCU 구조를 이해하려고 했습니다. 처음 봤을 때는 bus matrix, AHB, APB, NVIC, DMA가 한꺼번에 보여서 어렵게 느껴졌습니다. 그래서 처음에는 이 구조를 참고해서 bus와 peripheral을 잡으려고 했습니다.

하지만 실제 구현에서는 STM32F103을 그대로 따라간 것이 아니라, 구조를 참고한 뒤 core, bus, peripheral, DMA 구조를 직접 설계했습니다. 특히 AHB/APB 중심에서 출발했지만, RGB byte stream을 처리하기 위해 AXI-Lite, APB, AXI-Stream을 역할별로 나누는 방향으로 바꿨습니다.

핵심 이해

- STM32F103 그림은 "참고 구조"이지 그대로 구현한 구조가 아니다.
- 참고한 점: CPU, interrupt controller, DMA, peripheral bus가 있는 MCU-style system 구성.
- 바꾼 점: Cortex-M/NVIC/AHB bus를 그대로 쓰지 않고, custom RV32I core + AXI-Lite/APB + AXI-Stream DMA로 재구성.

코드 공부 위치

위치	볼 내용
<code>diagrams/stm32f103_reference.png</code>	slide 4에 들어간 원본 참고 그림
<code>final_source/src/bus/ahb</code>	이전 AHB 참고 구조 흔적
<code>Project/risc_axi/src/bus/axi</code>	현재 AXI-Lite 구조
<code>Project/risc_axi/src/bus/apb</code>	APB peripheral subsystem
<code>final_source/AXI_STREAM_HANDOFF.md</code>	AHB 이후 AXI/stream 확장 방향 메모

질문 대비

Q. STM32F103이랑 같은 구조인가?

A. 아니다. 데이터시트의 전체 MCU 구성을 참고했지만, 구현은 RV32I core, AXI-Lite/APB, AXI-Stream DMA 중심으로 재설계했다.

5. Slide 5 - SocTop Architecture

발표 대본

이 slide는 전체 SoC의 본체인 `SocTop` 구조입니다. 왼쪽은 custom RV32I core와 instruction fetch용 local ROM입니다. CPU의 DBus는 AXI-Lite master adapter를 거쳐 AXI-Lite interconnect로 들어가고, 이 interconnect는 ROM window, SRAM, APB bridge 세 영역으로 주소를 라우팅합니다.

APB 쪽에는 UART, SPI, GPIO, timer, PLIC, DMA 같은 peripheral이 연결됩니다. 특히 DMA는 APB register로 제어되지만, 실제 payload는 AXI-Stream 형태로 UART RX와 SPI TX 사이를 흐릅니다.

ROM이 두 개처럼 보일 수 있는데, `IcodeLocalRom`은 instruction fetch fast path이고, `AxiLiteRom`은 DBus에서 ROM window를 볼 수 있게 만든 AXI-Lite slave입니다. 발표할 때는 "프로그램이 두 개"라기보다 fetch path와 data bus visible window를 분리했다고 말하면 됩니다.

핵심 이해

- `SocTop`은 core, instruction ROM, AXI-Lite fabric, APB subsystem을 묶는 중심 module이다.
- instruction fetch는 local ROM path로 빠르게 분리되어 있다.
- DBus는 AXI-Lite로 memory-mapped control access를 수행한다.
- APB는 저속 peripheral register interface로 유지한다.
- DMA는 APB로 설정하지만 stream payload는 APB bus를 타지 않는다.

코드 공부 위치

위치	볼 내용
<code>Project/risc_axi/src/soc/SocTop.sv:6</code>	파일 상단 역할 설명
<code>Project/risc_axi/src/soc/SocTop.sv:177</code>	<code>Rv32Core</code> 연결
<code>Project/risc_axi/src/soc/SocTop.sv:211</code>	<code>IcodeLocalRom</code> 연결
<code>Project/risc_axi/src/soc/SocTop.sv:228</code>	<code>AxiLiteMasterAdapter</code> 연결
<code>Project/risc_axi/src/soc/SocTop.sv:261</code>	<code>AxiLiteInterconnect1x3</code> 연결
<code>Project/risc_axi/src/soc/SocTop.sv:334</code>	<code>AxiLiteRom</code> 연결
<code>Project/risc_axi/src/soc/SocTop.sv:359</code>	<code>AxiLiteSram</code> 연결
<code>Project/risc_axi/src/soc/SocTop.sv:388</code>	<code>AxiLiteToApbBridge</code> 연결
<code>Project/risc_axi/src/soc/SocTop.sv:419</code>	<code>ApbSubsystem</code> 연결
<code>diagrams/anti_fpgatop_block_diagram.svg</code>	slide 5 구조도

질문 대비

Q 왜 ROM이 두 개처럼 보이냐?

A. instruction fetch용 local ROM과 DBus-visible AXI-Lite ROM window가 분리되어 있어서 그렇다. 발표에서는 "동일 firmware image를 instruction path와 data-visible window 관점에서 나눈 것"으로 설명하면 된다.

6. Slide 6 - AXI-Lite Fabric & APB Bridge

발표 대본

CPU의 DBus는 AXI-Lite master adapter를 거쳐 하나의 AXI-Lite master로 들어갑니다. 그 다음 `AxiLiteInterconnect1x3` 가 주소를 보고 ROM, SRAM, peripheral window 중 하나로 라우팅합니다.

Peripheral window로 들어온 access는 `AxiLiteToApbBridge` 에서 APB transaction으로 변환됩니다. 이후 `ApbSubsystem` 내부 address decoder가 UART, SPI, GPIO, DMA, PLIC 같은 peripheral register block 중 하나를 선택합니다.

핵심 이해

- AXI-Lite는 memory-mapped register access에 적합하다.
- 이 프로젝트는 one-master 구조다. master는 core DBus 하나다.
- interconnect는 3개 slave window를 가진다: ROM, SRAM, peripheral.
- APB bridge는 AXI-Lite protocol과 APB protocol 사이의 timing/handshake 변환 계층이다.

코드 공부 위치

위치	볼 내용
<code>Project/risc_axi/src/bus/axi/AxiLiteMasterAdapter.sv:18</code>	core DBus -> AXI-Lite 변환 module
<code>Project/risc_axi/src/bus/axi/AxiLiteInterconnect1x3.sv:5</code>	interconnect 역할 설명
<code>Project/risc_axi/src/bus/axi/AxiLiteInterconnect1x3.sv:92</code>	ROM/SRAM/PERIPH select enum
<code>Project/risc_axi/src/bus/axi/AxiLiteInterconnect1x3.sv:104</code>	주소 decode: ROM
<code>Project/risc_axi/src/bus/axi/AxiLiteInterconnect1x3.sv:107</code>	주소 decode: SRAM
<code>Project/risc_axi/src/bus/axi/AxiLiteInterconnect1x3.sv:110</code>	주소 decode: peripheral
<code>Project/risc_axi/src/bus/axi/AxiLiteToApbBridge.sv:18</code>	AXI-Lite -> APB bridge module
<code>Project/risc_axi/src/bus/apb/ApbSubsystem.sv:15</code>	APB subsystem 시작
<code>diagrams/anti_bus_block_diagram.svg</code>	slide 6 구조도

질문 대비

Q. 왜 APB를 남겼나?

A. UART/SPI/GPIO/timer 같은 저속 register peripheral은 APB 구조가 단순하고 충분하다. AXI-Lite는 상위 control fabric으로 쓰고, peripheral 내부는 APB로 유지하는 것이 설계가 깔끔하다.

7. Slide 7 - FPGA Top & Memory Map

발표 대본

SocFpgaTop 은 실제 FPGA board pin, 100 MHz clock, reset, switch, LED, UART, SPI pin을 SoC에 연결하는 wrapper입니다. 반면 SocTop 은 core, bus, peripheral, DMA가 들어있는 SoC 본체입니다.

Memory map은 크게 세 영역입니다. 0x0000_0000 은 ROM window, 0x2000_0000 은 SRAM, 0x4000_0000 은 peripheral window입니다. PLIC은 peripheral 영역 안에서도 0x400F_0000 에 배치했습니다.

핵심 이해

- SocFpgaTop 은 board-level wrapper다.
- SocTop 은 reusable SoC 본체다.
- software가 보는 주소는 ROM, SRAM, peripheral 세 영역으로 나뉜다.
- interrupt flag 같은 software 상태는 SRAM 영역에 둔다.
- peripheral window 안에서는 APB subsystem이 PADDR[19:16] 기준으로 timer, GPIO, SPI, I2C, UART, DMA, PLIC를 64KB 단위로 나눈다.

APB 세부 주소

Block	Base 주소	역할
Timer	0x4000_0000	timer / timer IRQ
GPIOA	0x4001_0000	switch / LED
GPIOB	0x4002_0000	GPIO 확장
SPI	0x4003_0000	SPI master
I2C	0x4004_0000	I2C master
UART	0x4005_0000	PC serial
DMA	0x4006_0000	AXI-Stream DMA control
GPIOC	0x4007_0000	GPIO 확장
PLIC-lite	0x400F_0000	external interrupt controller

코드 공부 위치

위치	볼 내용
Project/risc_axi/src/soc/SocFpgaTop.sv:15	FPGA top wrapper module
Project/risc_axi/src/soc/SocTop.sv:17	SoC 본체 module
Project/risc_axi/src/bus/axi/axi_lite_pkg.sv	ROM/SRAM/PERIPH base/mask 정의
Project/risc_axi/src/bus/apb/ApbSubsystem.sv:105	APB 세부 peripheral decode 시작
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:23	GPIO/SPI/UART/DMA/PLIC/SRAM base address
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:53	SRAM flag offset

질문 대비

Q. memory map이 왜 중요한가?

A. assembly ROM이 peripheral을 제어할 때 모든 접근이 address 기반 load/store로 이루어진다. 따라서 주소 맵이 software와 hardware 사이의 계약이다.

8. Slide 8 - AXI-Lite vs AXI-Stream

발표 대본

AXI-Lite와 AXI-Stream의 차이는 주소가 있느냐 없느냐로 생각하면 이해하기 쉽습니다. AXI-Lite는 CPU가 특정 peripheral register 주소에 값을 쓰거나 읽는 control path입니다. 예를 들어 DMA length를 설정하거나 PLIC enable을 쓰는 동작입니다.

반면 AXI-Stream은 주소 없이 TVALID, TREADY, TDATA handshake로 byte가 순서대로 흐르는 data path입니다. 이 프로젝트에서는 UART RX stream이 DMA 내부 buffer로 들어가고, 다시 DMA가 SPI TX stream으로 내보내는 데 사용됩니다.

핵심 이해

- AXI-Lite: AW/W/B, AR/R channel을 가진 memory-mapped control protocol.
- AXI-Stream: TVALID/TREADY/TDATA 로 흐르는 payload protocol.
- DMA는 두 세계를 연결한다. APB register로 제어되고, stream interface로 data를 받거나 보낸다.

코드 공부 위치

위치	볼 내용
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:34	slave stream 입력 iS_TDATA/iS_TVALID/oS_TREADY
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:38	master stream 출력 oM_TDATA/oM_TVALID/iM_TREADY
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:127	oS_TREADY assignment
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:128	oM_TDATA/oM_TVALID assignment
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:131	stream handshake detect
Project/risc_axi/src/bus/axi/AxiLiteInterconnect1x3.sv	AXI-Lite address routing

질문 대비

Q. APB와 AXI-Lite는 둘 다 register access인데 왜 둘 다 쓰나?

A. AXI-Lite는 SoC 상위 control fabric에 쓰고, APB는 peripheral 내부 register block을 단순하게 유지하기 위해 쓴다. APB는 저속 peripheral에 충분하고 구현이 간단하다.

9. Slide 9 - AXI Path Summary

발표 대본

이 그림은 control path와 data path를 한 번에 정리한 slide입니다. CPU가 register를 설정하는 경로는 AXI-Lite와 APB를 따라갑니다. 하지만 RGB byte가 실제로 움직이는 경로는 UART RX에서 DMA buffer를 거쳐 SPI TX로 가는 stream path입니다.

따라서 CPU가 payload를 매번 읽고 쓰는 것이 아니라, CPU는 register 설정과 interrupt 처리만 담당하고 data는 별도 hardware path로 이동합니다.

핵심 이해

- 한 시스템 안에 control path와 data path가 동시에 있다.
- APB register access는 "설정"이고, AXI-Stream handshake는 "전송"이다.
- 이 slide는 8번 slide의 표를 실제 프로젝트 구조에 대입한 그림이다.

코드 공부 위치

위치	볼 내용
<code>diagrams/axi_path_summary.svg</code>	slide 9 그림
<code>Project/risc_axi/src/bus/apb/ApbSubsystem.sv:249</code>	<code>ApbAxiStreamDma</code> instance
<code>Project/risc_axi/src/bus/apb/ApbSubsystem.sv:273</code>	<code>ApbPlicLite</code> instance
<code>Project/risc_axi/src/bus/apb/ApbUart.sv:17</code>	UART APB wrapper
<code>Project/risc_axi/src/bus/apb/ApbSpi.sv:16</code>	SPI APB wrapper

질문 대비

Q. data stream이 APB를 타지 않는다는 말은 무슨 뜻인가?

A. APB는 DMA control register를 설정할 때만 사용한다. RGB byte payload는 DMA의 stream port를 통해 UART/SPI와 handshake한다.

10. Slide 10 - RV32 Pipeline Core

발표 대본

core는 RV32I 기반 5-stage pipeline 구조입니다. IF, ID, EX, MEM, WB 단계로 나뉘고, hazard와 forwarding unit을 통해 기본적인 pipeline 제어를 합니다. SoC 관점에서 중요한 점은 이 core가 instruction fetch path와 data access path를 분리해서 사용한다는 것입니다.

Interrupt가 들어오면 trap controller와 CSR file이 관여합니다. mepc 에는 trap 전 PC가 저장되고, mcause 에는 원인이 기록됩니다. 이후 PC는 mtvec 에 설정된 0x80 trap entry로 이동합니다.

핵심 이해

- pipeline 자체보다 발표에서는 SoC에 연결되는 core interface가 중요하다.
- IBus : instruction fetch, IcodeLocalRom 으로 연결.
- DBus : load/store MMIO, AXI-Lite master adapter로 연결.
- interrupt/trap은 CSR와 TrapController가 core 내부 제어 흐름을 바꾼다.

코드 공부 위치

위치	볼 내용
Project/risc_axi/src/core/pipeline/Rv32Core.sv:19	core top module
Project/risc_axi/src/core/pipeline/FetchStage.sv	instruction fetch
Project/risc_axi/src/core/pipeline/DecodeStage.sv	decode
Project/risc_axi/src/core/pipeline/ExecuteStage.sv	ALU/branch
Project/risc_axi/src/core/pipeline/MemoryStage.sv	memory/MMIO access
Project/risc_axi/src/core/pipeline/WritebackStage.sv	writeback
Project/risc_axi/src/core/pipeline/HazardUnit.sv	stall/flush 조건
Project/risc_axi/src/core/pipeline/ForwardingUnit.sv	forwarding
Project/risc_axi/src/core/pipeline/TrapController.sv:15	trap control
Project/risc_axi/src/core/pipeline/CsrFile.sv:15	CSR storage/update
diagrams/anti_core_block_diagram.svg	slide 10 core diagram

질문 대비

Q. core 설명을 얼마나 깊게 해야 하나?

A. 발표 시간이 짧으면 pipeline 세부 구현보다 SoC 연결, CSR/trap, interrupt 진입을 중심으로 말하는 것이 좋다.

11. Slide 11 - Peripheral Architecture

발표 대본

Peripheral 쪽은 APB slave interface를 공통으로 사용합니다. CPU가 APB register를 통해 control/status 값을 읽고 쓰면, 각 peripheral 내부 FSM이 실제 동작을 수행합니다. UART는 RX/TX shift와 oversampling, SPI는 clock generation과 shift 동작, I2C는 SCL timing과 start/stop/ack FSM을 가집니다.

이 slide는 peripheral들이 모두 독립 module이지만, SoC에서는 APB register block으로 통일되어 있다는 점을 보여줍니다.

핵심 이해

- APB slave interface는 peripheral register 접근 방식의 통일 계층이다.
- UART/SPI/I2C/GPIO는 각각 내부 FSM이 다르지만 CPU 입장에서는 load/store register 접근으로 보인다.
- RGB 시나리오에서 특히 중요한 peripheral은 UART, SPI, DMA, PLIC이다.

코드 공부 위치

위치	볼 내용
Project/risc_axi/src/bus/apb/ApbSubsystem.sv:15	APB subsystem
Project/risc_axi/src/bus/apb/ApbUart.sv:17	UART APB wrapper
Project/risc_axi/src/peripheral/uart/UartRx.sv:18	UART RX
Project/risc_axi/src/peripheral/uart/UartTx.sv:18	UART TX
Project/risc_axi/src/bus/apb/ApbSpi.sv:16	SPI APB wrapper
Project/risc_axi/src/peripheral/spi/SpiMaster.sv:19	SPI master
Project/risc_axi/src/bus/apb/ApbTimer.sv:15	timer
Project/risc_axi/src/bus/apb/ApbGpio.sv	APB GPIO
Project/risc_axi/src/peripheral/gpio/Gpio.sv:15	GPIO core
diagrams/anti_peri_block_diagram.svg	slide 11 peripheral diagram

질문 대비

Q. APB slave와 peripheral core는 왜 나누나?

A. APB wrapper는 CPU register interface를 담당하고, peripheral core는 실제 serial timing/FSM을 담당한다. 이렇게 나누면 bus protocol과 동작 FSM을 분리해서 이해할 수 있다.

12. Slide 12 - PLIC

발표 대본

PLIC은 여러 interrupt source 중 어떤 source를 CPU에 전달할지 결정하는 interrupt controller입니다. UART, SPI, DMA 같은 peripheral에서 raw IRQ가 들어오면 gateway가 1회성 request로 바꾸고, pending bit에 저장합니다. 그중 enable되어 있고 threshold보다 priority가 높은 source만 claim 후보가 됩니다.

중요한 점은 PLIC이 handler 주소를 직접 주지 않는다는 것입니다. PLIC은 CPU에게 source ID만 알려줍니다. 그 다음 handler 선택은 trap handler 안의 software dispatch가 처리합니다.

핵심 이해

- raw IRQ level -> gateway request pulse -> pending latch.
- enable, priority, threshold 조건으로 claimable source 결정.
- `oExternalIrq` 가 core로 들어가 machine external interrupt를 만든다.
- CPU는 `CLAIM` 을 읽어서 source ID를 얻고, 처리 후 `COMPLETE` 에 같은 ID를 쓴다.

코드 공부 위치

위치	볼 내용
<code>Project/risc_axi/src/bus/apb/ApbPlicLite.sv:8</code>	PLIC 동작 설명 주석
<code>Project/risc_axi/src/bus/apb/ApbPlicLite.sv:47</code>	ENABLE/PENDING/CLAIM/COMPLETE/THRESHOLD offset
<code>Project/risc_axi/src/bus/apb/ApbPlicLite.sv:112</code>	enabled pending 계산
<code>Project/risc_axi/src/bus/apb/ApbPlicLite.sv:124</code>	claim 후보 선택 loop
<code>Project/risc_axi/src/bus/apb/ApbPlicLite.sv:139</code>	<code>oExternalIrq</code> 생성
<code>Project/risc_axi/src/bus/apb/ApbPlicLite.sv:172</code>	CLAIM read는 ID 반환만 함
<code>Project/risc_axi/src/bus/apb/ApbPlicLite.sv:211</code>	COMPLETE write 처리
<code>Project/risc_axi/src/bus/apb/ApbPlicGateway.sv:17</code>	gateway source 수
<code>diagrams/anti_plic_block_diagram.svg</code>	slide 12 PLIC diagram

질문 대비

Q. 일반 RISC-V PLIC과 완전히 같은가?

A. project용 PLIC-lite다. 특히 이 구현은 CLAIM read에서 pending을 지우지 않고, COMPLETE write에서 pending/gateway를 정리한다. ROM handler 타이밍을 안정적으로 맞추기 위한 단순화다.

13. Slide 13 - Interrupt Path & CSR

발표 대본

DMA done 같은 interrupt 이벤트가 발생하면 PLIC이 external IRQ를 core에 전달합니다. core 내부에서는 trap controller가 현재 실행 흐름을 멈추고 CSR file에 상태를 저장합니다. `mepc`에는 trap 발생 전 PC가 저장되고, `mcause`에는 interrupt 여부와 cause가 기록됩니다. 이후 PC는 `mtvec`에 설정한 `0x80` trap entry로 이동합니다.

이번 firmware에서는 `mtvec=0x80`, `mie.MEIE=1`, `mstatus.MIE=1`을 설정해서 machine external interrupt를 받을 수 있게 했습니다.

핵심 이해

- `mtvec`: trap entry base address.
- `mie.MEIE`: machine external interrupt enable.
- `mstatus.MIE`: global interrupt enable.
- `mcause`: interrupt인지 exception인지, 원인이 무엇인지 기록.
- `mepc`: trap 전 PC 저장.

PLIC / TrapController / CSR File 역할 구분

블록	위치	핵심 역할	발표용 한 문장
PLIC	core 밖 APB peripheral	여러 interrupt source 중 CPU가 처리할 source ID 선택	"PLIC은 interrupt 접수창구처럼 어떤 source가 들어왔는지 번호를 골라준다."
TrapController	core 내부 제어부	interrupt/exception 발생 시 PC를 <code>mtvec</code> 으로 redirect	"TrapController는 평소 실행 흐름을 멈추고 trap handler로 PC를 꺾는다."
CSR File	core 내부 특수 register	<code>mtvec</code> , <code>mepc</code> , <code>mcause</code> , <code>mie</code> , <code>mstatus</code> 같은 trap 상태와 enable 저장	"CSR File은 어디서 멈췄고 왜 trap에 들어왔는지 기록하는 상태 저장부다."

셋의 관계는 아래 순서로 이해하면 된다.

```
DMA_DONE 발생
-> PLIC가 claim ID 6을 선택
-> core로 external IRQ 전달
-> TrapController가 PC를 mtvec(0x80)으로 redirect
-> CSR File에 mepc, mcause 등 상태 기록
-> firmware trap_entry가 PLIC CLAIM을 읽고 irq_dma_done으로 dispatch
-> handler 처리 후 PLIC COMPLETE write
-> mret으로 원래 흐름 복귀
```

헛갈리면 안 되는 포인트는 `PLIC`이 handler 주소를 직접 주지 않는다는 것이다. PLIC은 source ID만 주고, 실제 handler 선택은 ROM의 software vector table이 한다.

코드 공부 위치

위치	볼 내용
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:82	trap_entry
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:105	csrr t0, mcause
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:314	CSR 설정 주석
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:319	csrw mtvec, t0
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:322	csrw mie, t0
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:324	csrs mstatus, t0
Project/risc_axi/src/core/pipeline/CsrFile.sv:15	CSR file module
Project/risc_axi/src/core/pipeline/TrapController.sv:15	trap controller module

질문 대비

Q. mtvec=0x80 은 누가 정하나?

A. firmware가 CSR write로 설정한다. ROM layout에서 0x80 위치에 trap_entry 를 배치했고, reset code가 mtvec 을 그 주소로 설정한다.

14. Slide 14 - Software Vector Table vs ARM NVIC

발표 대본

ARM NVIC와 현재 RISC-V firmware 설계의 차이는 handler 주소를 누가 선택하느냐입니다. ARM NVIC에서는 interrupt number가 들어오면 hardware가 vector table에서 handler 주소를 읽고 PC를 이동시킵니다.

반면 현재 설계는 모든 trap이 `mtvec=0x80` direct entry로 들어옵니다. 그 안에서 software가 PLIC CLAIM ID를 읽고, ROM 안의 software vector table에서 handler 주소를 골라 `jalr` 로 분기합니다. routine 자체는 둘 다 ROM/Flash code 영역에 있지만, dispatch 주체가 hardware냐 software냐가 다릅니다.

핵심 이해

- ARM NVIC: hardware vector table lookup.
- 현재 RISC-V: direct trap entry -> software claim -> software table lookup -> `jalr` .
- PLIC claim ID는 handler 주소가 아니라 source ID다.
- 이번 ROM에서는 ID 6 = DMA_DONE, ID 7 = DMA_ERROR를 handler에 연결한다.

코드 공부 위치

위치	볼 내용
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:111</code>	<code>irq_external</code> , PLIC claim 처리
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:155</code>	software vector dispatch 설명 주석
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:161</code>	<code>la t5, irq_vector_table</code>
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:169</code>	<code>irq_vector_table</code> 시작
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:176</code>	ID 6 -> <code>irq_dma_done</code>
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:177</code>	ID 7 -> <code>irq_dma_error</code>
<code>SW_TRAP_PLIC_VISUAL_GUIDE.md</code>	PLIC/vector 설명 참고
<code>final_source/SW_VECTOR_TABLE_NOTE.md</code>	이전 vector table 설명 참고

질문 대비

Q. ARM NVIC도 결국 handler는 ROM에 있는 것 아닌가?

A. 맞다. handler routine은 둘 다 ROM/Flash의 code다. 차이는 ARM은 hardware가 vector table에서 handler PC를 직접 고르고, 현재 설계는 software가 PLIC claim ID를 보고 handler를 고른다는 점이다.

15. Slide 15 - PLIC-lite Claim/Complete Flow

발표 대본

PLIC-lite의 처리 순서는 다섯 단계로 볼 수 있습니다. 먼저 UART/SPI/DMA 같은 raw IRQ source가 들어오고, gateway가 complete 전 중복 request를 막으면서 request pulse를 만듭니다. 이 request가 pending에 latch 되고, enable과 priority 조건을 통과하면 external IRQ가 core로 올라갑니다.

trap handler는 PLIC CLAIM register를 읽어서 source ID를 얻습니다. handler가 실제 처리를 끝낸 뒤에는 COMPLETE register에 같은 ID를 써서 PLIC 내부 pending과 gateway 상태를 정리합니다.

현재 RGB DMA 시나리오에서 실제로 handler에 연결된 source는 ID 6 `DMA_DONE`, ID 7 `DMA_ERROR` 입니다.

핵심 이해

- CLAIM read: 현재 처리할 source ID를 읽는다.
- COMPLETE write: 처리 완료를 알리고 pending/gateway block을 해제한다.
- 이 구현은 CLAIM read만으로 pending을 지우지 않는다.
- software vector table이 claim ID를 handler 주소로 바꾼다.

코드 공부 위치

위치	볼 내용
Project/risc_axi/src/bus/apb/ApbPlicLite.sv:15	CLAIM read side effect 없음 설명
Project/risc_axi/src/bus/apb/ApbPlicLite.sv:24	COMPLETE에서 정리한다는 설명
Project/risc_axi/src/bus/apb/ApbPlicLite.sv:47	PLIC register offset
Project/risc_axi/src/bus/apb/ApbPlicLite.sv:144	complete pulse 조건
Project/risc_axi/src/bus/apb/ApbPlicLite.sv:199	pending latch/complete clear
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:65	DMA done/error IRQ ID
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:111	CLAIM read path
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:183	DMA done handler
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:196	DMA error handler

질문 대비

Q. COMPLETE를 안 쓰면 어떻게 되나?

A. pending/gateway 상태가 정리되지 않아 같은 source가 계속 active/block 상태로 남을 수 있다. 그래서 handler는 처리 후 반드시 같은 ID를 COMPLETE에 써야 한다.

16. Slide 16 - End-to-End Image Transfer

발표 대본

이제 실제 application scenario입니다. PC가 RGB image byte를 UART로 master FPGA에 보냅니다. master FPGA 내부에서는 UART RX stream이 DMA buffer로 들어가고, 한 frame이 다 들어오면 DMA done interrupt가 발생합니다. 이후 firmware가 TX phase로 전환해서 DMA buffer의 byte를 SPI TX stream으로 내보냅니다.

slave FPGA는 SPI로 받은 byte를 다시 UART로 PC에 돌려보내고, PC는 원본 image와 수신 image를 비교합니다. 그래서 이 slide는 "SoC 내부 구조가 실제 데이터 전송 시나리오에서 어떻게 쓰이는지"를 보여줍니다.

핵심 이해

- 전체 경로: PC -> UART -> master FPGA -> DMA buffer -> SPI -> slave FPGA -> UART -> PC.
- master FPGA의 CPU는 phase 전환과 interrupt 처리를 담당한다.
- 실제 byte payload는 DMA/stream path를 통해 움직인다.
- slave는 SPI RX byte를 받아 UART TX로 돌려주는 endpoint 역할이다.

코드 공부 위치

위치	볼 내용
<code>diagrams/end_to_end_image_transfer.svg</code>	slide 16 end-to-end 그림
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:326</code>	master firmware main loop
<code>Project/risc_axi/src/bus/apb/ApbAxisStreamDma.sv:67</code>	DMA internal byte buffer
<code>Project/slave_test/src/Top.sv:3</code>	slave FPGA top
<code>Project/slave_test/src/slave/SpiSlaveByteRx.sv:3</code>	SPI slave byte RX
<code>Project/slave_test/src/stream/ByteFifo.sv:3</code>	slave byte FIFO
<code>Project/slave_test/src/peripheral/uart/UartTx.sv:3</code>	slave UART TX
<code>python_comport/src/send_image.py</code>	PC image send
<code>python_comport/src/receive_image.py</code>	PC image receive
<code>python_comport/src/transfer_compare.py</code>	송수신 비교

질문 대비

Q. 왜 slave FPGA가 필요한가?

A. master가 SPI로 내보낸 pixel byte가 외부 pin을 통해 실제로 전달되는지 확인하려면 반대편에서 받아서 다시 PC로 돌려주는 endpoint가 필요하다.

17. Slide 17 - AXI-Stream DMA

발표 대본

ApbAxiStreamDma 는 APB로 제어되고 AXI-Stream으로 data를 옮기는 block입니다. CPU는 LEN_BYTES , BUF_ADDR , CTRL 같은 register만 설정합니다. DMA 내부에는 16 KB byte buffer가 있고, RGB888 64x64 frame은 12,288 byte라서 한 frame을 담을 수 있습니다.

RX phase에서는 UART RX stream의 byte가 buffer에 저장되고, TX phase에서는 buffer의 byte가 SPI TX stream으로 출력됩니다. 마지막 byte까지 처리하면 DMA done sticky와 interrupt가 올라가고, firmware handler가 SRAM flag를 세워 main loop가 다음 phase로 넘어갑니다.

핵심 이해

- buffer size: P_BUFFER_BYTES = 16384 .
- RGB888 64x64: $64 * 64 * 3 = 12288$.
- CTRL=3 : start + IRQ enable + RX direction.
- CTRL=7 : start + IRQ enable + TX direction.
- LEN_BYTES 는 byte 단위 전송 길이이다.
- BUF_ADDR 는 DMA internal buffer offset이다.

코드 공부 위치

위치	볼 내용
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:17	DMA module 시작
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:18	16 KB buffer parameter
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:44	CTRL , STATUS , LEN_BYTES , BUF_ADDR offsets
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:67	internal byte buffer
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:95	start pulse
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:106	range check
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:119	receive mode buffer write data
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:151	buffer write
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:154	transmit mode buffer read
Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:58	default IMAGE_BYTES=12288

질문 대비

Q. SRAM을 쓰는 DMA인가?

A. 현재 RGB stream DMA slide 기준으로는 DMA 내부 16 KB buffer를 사용한다. CPU-visible SRAM은 interrupt flag/debug state용으로 쓰인다.

18. Slide 18 - UART-to-SPI DMA Stream Flow

발표 대본

이 slide는 firmware 실행 흐름을 한눈에 보여주는 그림입니다. reset 후 firmware는 GPIO, UART, SPI, DMA, PLIC를 초기화하고, CSR에서 `mtvec`, `mie`, `mstatus`를 설정합니다. 이후 main loop에서 먼저 RX phase를 시작합니다.

RX phase에서는 PC에서 들어온 UART byte stream이 DMA buffer에 쌓입니다. 12,288 byte가 다 들어오면 DMA done interrupt가 발생하고, trap handler가 PLIC claim ID 6을 읽어서 `irq_dma_done` handler로 분기합니다. handler는 SRAM의 `FLAG_DONE`을 set하고, main loop는 그 flag를 보고 TX phase로 넘어갑니다.

TX phase에서는 같은 buffer를 SPI TX stream으로 내보냅니다. 마지막 byte가 나가면 다시 DMA done interrupt가 발생하고, main loop는 다음 frame을 기다리기 위해 처음으로 돌아갑니다.

핵심 이해

- 이 slide는 software의 phase machine이다.
- RX와 TX 모두 DMA done/error interrupt를 쓴다.
- main loop는 DMA status register를 계속 polling하지 않고 SRAM flag를 본다.
- SRAM flag는 interrupt handler가 set한다.

코드 공부 위치

위치	볼 내용
<code>diagrams/rgb_irq_rom_flow_v2.svg</code>	slide 18 flow diagram
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:247</code>	<code>reset_main</code>
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:269</code>	SRAM flag 초기화
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:309</code>	PLIC DMA mask enable
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:326</code>	<code>main_loop</code>
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:345</code>	<code>wait_rx_irq</code>
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:393</code>	<code>wait_tx_irq</code>
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:183</code>	<code>irq_dma_done</code>
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S:196</code>	<code>irq_dma_error</code>
<code>RISC_AXI_ASM_PRESENTATION_GUIDE.md</code>	ASM 발표 설명 참고

질문 대비

Q. main loop가 polling을 안 한다는 말이 맞나?

A. DMA status register polling은 하지 않는다. 대신 handler가 SRAM flag를 set하고 main loop가 그 flag를 기다린다. 즉 hardware completion은 interrupt로 받고, software phase wait는 SRAM flag로 단순화했다.

19. Slide 19 - TB Waveform Slot: DMA to SPI Stream

발표 대본

이 slide에는 나중에 DMA to SPI stream 파형을 넣으면 됩니다. 보여줄 포인트는 CPU가 `DMA_CTRL=7` 을 쓰는 순간 TX phase가 시작되고, DMA의 `M_TVALID/TDATA` 와 SPI 쪽 `TREADY` 가 handshake하면서 byte가 하나씩 전달되는 것입니다.

마지막 byte 이후에는 DMA done sticky가 set되고, PLIC source ID 6으로 interrupt가 전달되는 흐름을 같이 보여주면 좋습니다.

핵심 이해

- waveform에서 핵심 신호: `oM_TVALID` , `iM_TREADY` , `oM_TDATA` , `oM_TLAST` , DMA done IRQ.
- TX phase는 buffer -> SPI 방향이다.
- 마지막 byte 후 done interrupt가 발생해야 한다.

코드 공부 위치

위치	볼 내용
Project/risc_axi/tb/soc/tb_SocTopRgbIrqSpiWave.sv	SPI wave용 SoC TB
Project/risc_axi/tools/run_rgb_irq_spi_wave_sim.tcl	wave sim 실행 TCL
Project/risc_axi/tools/save_rgb_irq_spi_wave_wcfg.tcl	waveform config
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:128	master stream output
Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv:132	master stream handshake
Project/risc_axi/src/peripheral/spi/SpiMaster.sv:19	SPI master

질문 대비

Q. 파형에서 반드시 보여야 하는 것은?

A. `DMA_CTRL=7` 이후 valid/ready handshake가 진행되고, byte count가 끝난 뒤 DMA done interrupt로 이어지는 순서다.

20. Slide 20 - TB Waveform Slot: SPI Pixel Pins

발표 대본

이 slide에는 SPI pin 레벨 파형을 넣으면 됩니다. 앞 slide가 내부 AXI-Stream handshake라면, 이 slide는 실제 외부 pin에서 pixel byte가 나가는지 보여주는 증거입니다. `spi_sck`, `spi_mosi`, `spi_cs` 를 보고, MOSI bit를 복원했을 때 expected pixel byte sequence와 맞는지 설명하면 됩니다.

핵심 이해

- stream handshake는 내부 data movement 증거다.
- SPI pin waveform은 외부 출력 증거다.
- byte 하나가 SCK 여러 cycle 동안 MOSI로 shift out된다.

코드 공부 위치

위치	볼 내용
<code>Project/risc_axi/src/peripheral/spi/SpiMaster.sv:19</code>	SPI master FSM
<code>Project/risc_axi/src/bus/apb/ApbSpi.sv:16</code>	APB SPI wrapper
<code>Project/risc_axi/tb/soc/tb_SocTopRgbIrqSpiWave.sv</code>	SPI pin wave 관찰 testbench
<code>Project/slave_test/src/slave/SpiSlaveByteRx.sv:3</code>	slave 쪽 SPI byte receiver
<code>Project/slave_test/tb/tb_TopSpiToUart.sv</code>	slave SPI -> UART TB

질문 대비

Q. 왜 내부 stream 파형과 SPI pin 파형을 둘 다 보여주나?

A. 내부 DMA가 data를 내보내는 것과, 실제 SPI pin으로 byte가 shift out되는 것은 서로 다른 검증 포인트다. 둘을 같이 보여주면 data path가 끊기지 않았음을 설명하기 쉽다.

21. Slide 21 - Demo: End-to-End Image Transfer

발표 대본

이 slide에는 실제 demo 영상이나 캡처를 넣으면 됩니다. 설명은 네 단계로 하면 됩니다. PC sender가 RGB image byte를 UART로 보내고, master FPGA가 UART RX에서 DMA buffer로 받은 뒤 SPI로 전달합니다. slave FPGA는 SPI RX byte를 UART TX로 PC에 되돌려 보내고, PC는 원본과 수신 결과를 비교합니다.

demo에서 가장 중요한 것은 "눈으로 보기 쉬운 결과"입니다. 원본 이미지, 수신 이미지, diff 이미지 또는 byte compare log를 같이 보여주면 좋습니다.

핵심 이해

- demo는 구조 설명의 최종 증거다.
- PC script는 image byte stream 생성/송신/수신/비교를 담당한다.
- FPGA 쪽은 master와 slave로 역할이 나뉜다.
- diff가 작거나 0이면 end-to-end byte path가 맞다는 뜻이다.

코드 공부 위치

위치	볼 내용
<code>python_comport/src/transfer_compare.py</code>	송수신 비교 main script
<code>python_comport/src/send_image.py</code>	image byte 송신
<code>python_comport/src/receive_image.py</code>	수신 image 복원
<code>python_comport/src/protocol.py</code>	PC 측 protocol/helper
<code>python_comport/outputs</code>	기존 실험 결과 이미지/로그
<code>Project/risc_axi/tools/program_rgb_irq_master_slave.tcl</code>	master/slave programming script
<code>Project/slave_test/src/Top.sv:3</code>	slave top

질문 대비

Q. demo에서 성공을 무엇으로 판단하나?

A. PC가 보낸 byte sequence와 slave를 거쳐 돌아온 byte sequence가 일치하는지, image mode라면 원본/수신/diff 이미지가 일치하는지로 판단한다.

22. Slide 22 - Result & Lessons Learned

발표 대본

이번 프로젝트에서는 처음에 STM32F103 데이터시트를 봤을 때 전체 구조가 잘 보이지 않아 어렵게 느껴졌습니다. 하지만 core, bus, peripheral, interrupt를 하나씩 나눠서 공부하고 직접 연결해 보면서 데이터시트의 블록들이 어떤 역할을 하는지 조금씩 이해할 수 있었습니다.

구현 결과로는 RV32I core를 AXI-Lite fabric, APB subsystem, AXI-Stream DMA와 통합했고, PLIC claim ID 기반 software vector dispatch로 interrupt phase 전환을 구현했습니다. 특히 ARM NVIC처럼 hardware가 vector table을 직접 읽는 방식과, RISC-V에서 `mtvec`, PLIC claim, software dispatch를 직접 구성하는 방식의 차이를 체감했습니다.

아쉬웠던 점은 linker, compiler, loader까지 설계하기에는 범위가 컸다는 점입니다. 그래서 이번에는 assembly를 ROM image로 직접 올리는 방식으로 진행했고, firmware를 바꿀 때 bitstream을 다시 만들어야 하는 불편함도 느꼈습니다.

핵심 이해

- 결과는 "core + bus + peripheral + DMA + interrupt + firmware" 통합이다.
- 느낀점은 솔직하게 말하는 것이 좋다.
- 어려웠던 점: 데이터시트 구조 이해, bus protocol, interrupt dispatch, ROM firmware 반복 build.
- 배운 점: MCU 구조의 큰 그림, control/data path 분리, CSR/PLIC interrupt 처리.

코드 공부 위치

위치	볼 내용
<code>Project/risc_axi/src/soc/SocTop.sv</code>	최종 통합 구조
<code>Project/risc_axi/src/bus/axi</code>	AXI-Lite fabric
<code>Project/risc_axi/src/bus/apb</code>	APB peripheral/DMA/PLIC
<code>Project/risc_axi/src/core/pipeline</code>	custom RV32I core
<code>Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S</code>	최종 ROM firmware
<code>Project/risc_axi/C/build_uart_dma_spi_rgb_irq_forward.ps1</code>	ASM -> ROM image build
<code>Project/risc_axi/src/timing_programs/uart_dma_spi_rgb_irq_forward.mem</code>	ROM에 들어가는 mem image

질문 대비

Q. 가장 크게 배운 점은?

A. MCU는 CPU만으로 동작하는 것이 아니라 bus, address map, interrupt controller, peripheral register, firmware가 함께 맞아야 한다는 점이다. 특히 data movement는 CPU가 직접 하는 일과 DMA/stream hardware가 해야 하는 일을 분리해서 생각해야 한다.

부록 A. 발표 전 30분 공부 순서

시간이 없으면 아래 순서로 보면 된다.

1. `slide.html` 을 한 번 열고 22장 제목 흐름을 확인한다.
2. `Project/risc_axi/src/soc/SocTop.sv` 에서 instance 연결만 본다.
3. `Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S` 에서 `reset_main` , `main_loop` , `trap_entry` , `irq_vector_table` , `irq_dma_done` 만 본다.
4. `Project/risc_axi/src/bus/apb/ApbAxisStreamDma.sv` 에서 stream port, register offset, buffer, handshake를 본다.
5. `Project/risc_axi/src/bus/apb/ApbPlicLite.sv` 에서 CLAIM/COMPLETE 동작을 본다.
6. `diagrams/end_to_end_image_transfer.svg` 와 `diagrams/rgb_irq_rom_flow_v2.svg` 로 발표 흐름을 입으로 연습한다.

부록 B. 한 문장 답변 모음

질문	짧은 답
왜 STM32F103을 참고했나	MCU 전체 구조를 이해하기 위한 기준으로 삼았다. 실제 구현은 RV32I/AXI/APB/Stream 구조로 재설계했다.
왜 AXI-Lite와 AXI-Stream을 나눴나	register control과 byte payload movement의 성격이 다르기 때문이다.
APB는 왜 남겼나	UART/SPI/GPIO 같은 저속 peripheral register에는 APB가 단순하고 충분하기 때문이다.
PLIC은 무엇을 하나	여러 interrupt source 중 CPU가 처리할 source ID를 고른다.
PLIC이 handler 주소를 주나	아니다. PLIC은 claim ID만 주고, handler 선택은 software vector table이 한다.
ARM NVIC와 차이는	NVIC는 hardware가 vector table lookup을 하고, 현재 설계는 software가 claim ID로 dispatch 한다.
DMA가 하는 일은	UART RX stream을 buffer에 받고, buffer의 byte를 SPI TX stream으로 내보낸다.
CPU 역할은	DMA register 설정, interrupt 처리, RX/TX phase 전환이다.
왜 ROM build가 불편했나	linker/compiler/loader 없이 ROM image를 bitstream에 넣는 방식이라 firmware 수정 때마다 bitstream 반복이 필요했다.

부록 C. 핵심 파일 빠른 색인

주제	파일
PPT 기준 HTML	slide.html
SoC top	Project/risc_axi/src/soc/SocTop.sv
FPGA wrapper	Project/risc_axi/src/soc/SocFpgaTop.sv
AXI-Lite adapter	Project/risc_axi/src/bus/axi/AxiLiteMasterAdapter.sv
AXI-Lite interconnect	Project/risc_axi/src/bus/axi/AxiLiteInterconnect1x3.sv
AXI-Lite to APB bridge	Project/risc_axi/src/bus/axi/AxiLiteToApbBridge.sv
APB subsystem	Project/risc_axi/src/bus/apb/ApbSubsystem.sv
AXI-Stream DMA	Project/risc_axi/src/bus/apb/ApbAxiStreamDma.sv
PLIC-lite	Project/risc_axi/src/bus/apb/ApbPlicLite.sv
PLIC gateway	Project/risc_axi/src/bus/apb/ApbPlicGateway.sv
RV32 core	Project/risc_axi/src/core/pipeline/Rv32Core.sv
CSR file	Project/risc_axi/src/core/pipeline/CsrFile.sv
Trap controller	Project/risc_axi/src/core/pipeline/TrapController.sv
RGB IRQ firmware	Project/risc_axi/C/runtime/uart_dma_spi_rgb_irq_forward.S
ROM mem image	Project/risc_axi/src/timing_programs/uart_dma_spi_rgb_irq_forward.mem
SPI wave TB	Project/risc_axi/tb/soc/tb_SocTopRgbIrqSpiWave.sv
mini RGB IRQ TB	Project/risc_axi/tb/soc/tb_SocTopRgbIrqMini.sv
core interrupt TB	Project/risc_axi/tb/core/tb_CoreInterruptPath.sv
slave top	Project/slave_test/src/Top.sv
slave SPI RX	Project/slave_test/src/slave/SpiSlaveByteRx.sv
PC transfer scripts	python_comport/src